Ablation : ConvNeXt-Tiny — SiLU | StochDepth | LayerNorm

Config: ImageNet- K pretrained | SiLU activation | StochDepth | LayerNorm **Model:** ConvNeXt

Dataset: Rice Leaf Disease Identification Dataset

. Colab Setup (jalankan sekali per session)

In [1]:
```
from google.colab import drive
drive.mount('/content/drive')
import os
DRIVE_DIR = '/content/drive/MyDrive/rice-convnext/05_SiLU_StochDepth_Laye
os.makedirs(DRIVE_DIR, exist_ok=True)
print(f'Drive folder ready: {DRIVE_DIR}')
```

```
Mounted at /content/drive
Drive folder ready: /content/drive/MyDrive/rice-convnext/
05_SiLU_StochDepth_LayerNorm
```

In [2]:
```
from google.colab import files
import os, shutil
print('Upload kaggle.json...')
files.upload()
os.makedirs('/root/.kaggle', exist_ok=True)
shutil.move('kaggle.json', '/root/.kaggle/kaggle.json')
os.chmod('/root/.kaggle/kaggle.json', 0o600)
print('Kaggle API ready!')
```

```
Upload kaggle.json...
```

Upload widget is only available when the cell has been executed in the current browser session. Pl
rerun this cell to enable.

```
Saving kaggle.json to kaggle.json
Kaggle API ready!
```

In [3]:
```
DATASET_DIR = '/content/drive/MyDrive/rice-convnext/dataset'
if os.path.exists(DATASET_DIR) and len(os.listdir(DATASET_DIR)) > 0:
    print('Dataset sudah ada, skip download.')
else:
    !pip install kaggle -q
    !kaggle datasets download -d wangxiaoqii/rice-leaf-disease-identifica
    !unzip -q /content/tmp/*.zip -d {DATASET_DIR}
    !rm -rf /content/tmp
    print('Dataset ready!')

import shutil
LOCAL_DATASET = '/content/dataset'
if not os.path.exists(LOCAL_DATASET):
    print('Copying dataset ke local SSD...')
    shutil.copytree(DATASET_DIR, LOCAL_DATASET)
    print('Done!')
else:
```

```
    print('Sudah ada di local.')
DATA_DIR = LOCAL_DATASET
```

```
Dataset sudah ada, skip download.
Copying dataset ke local SSD...
Done!
```

. Setup & Imports

In [4]:
```
import os, random, shutil
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from sklearn.metrics import classification_report, confusion_matrix
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms, models

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)
torch.backends.cudnn.deterministic = True

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {DEVICE}')
if torch.cuda.is_available():
    print(f'GPU: {torch.cuda.get_device_name(0)}')
```

```
Device: cuda
GPU: Tesla T4
```

. Configuration

In [5]:
```
DATA_DIR      = '/content/dataset'
DRIVE_DIR     = '/content/drive/MyDrive/rice-convnext/05_SiLU_StochDepth_
IMG_SIZE      = 224
BATCH_SIZE    = 64
NUM_EPOCHS    = 30
LR_INIT       = 1e-3
LR_FINAL      = 1e-6
WEIGHT_DECAY  = 0.05
NUM_WORKERS   = 2
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]
SAVE_PATH     = f'{DRIVE_DIR}/convnext_tiny_05_silu_stochdepth_layernorm.
EXP_NAME      = 'ConvNeXt-Tiny | SiLU | StochDepth | LayerNorm'
print(f'Experiment: {EXP_NAME}')
```

```
Experiment: ConvNeXt-Tiny | SiLU | StochDepth | LayerNorm
```

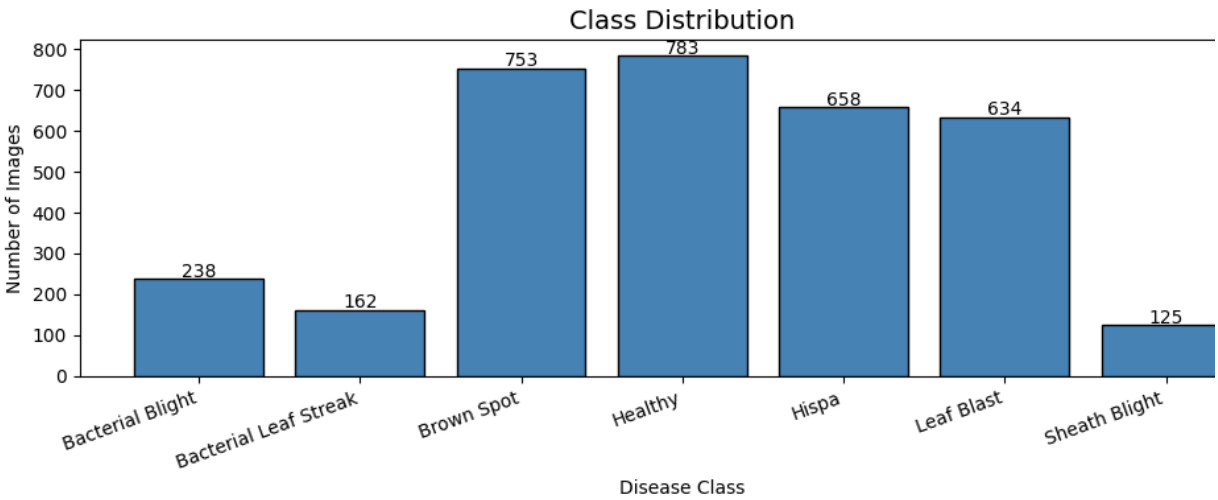
. Data Loading & Augmentation

In [6]:
```
train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(IMG_SIZE),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(30),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])
val_test_transforms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(IMG_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

full_dataset = datasets.ImageFolder(DATA_DIR)
CLASS_NAMES  = full_dataset.classes
NUM_CLASSES  = len(CLASS_NAMES)
print(f'Classes ({NUM_CLASSES}): {CLASS_NAMES}')
print(f'Total images: {len(full_dataset)}')

total   = len(full_dataset)
n_train = int(0.6 * total)
n_val   = int(0.2 * total)
n_test  = total - n_train - n_val

generator = torch.Generator().manual_seed(SEED)
train_set, val_set, test_set = random_split(
    full_dataset, [n_train, n_val, n_test], generator=generator
)

train_set.dataset.transform = train_transforms
val_set.dataset.transform   = val_test_transforms
test_set.dataset.transform  = val_test_transforms

train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True,
val_loader   = DataLoader(val_set,   batch_size=BATCH_SIZE, shuffle=False
test_loader  = DataLoader(test_set,  batch_size=BATCH_SIZE, shuffle=False
print(f'Train: {len(train_set)} | Val: {len(val_set)} | Test: {len(test_s
```

```
Classes (7): ['Bacterial Blight', 'Bacterial Leaf Streak', 'Brown Spot', 'Hea
'Hispa', 'Leaf Blast', 'Sheath Blight']
Total images: 3353
Train: 2011 | Val: 670 | Test: 672
```

In [7]:
```
labels_all = [full_dataset.targets[i] for i in range(len(full_dataset))]
counts = [Counter(labels_all)[i] for i in range(NUM_CLASSES)]
plt.figure(figsize=(10, 4))
bars = plt.bar(CLASS_NAMES, counts, color='steelblue', edgecolor='black')
plt.title('Class Distribution', fontsize=14)
plt.xlabel('Disease Class'); plt.ylabel('Number of Images')
plt.xticks(rotation=20, ha='right')
for bar, count in zip(bars, counts):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height()+5, str(cou
plt.tight_layout()
```

```
plt.savefig('class_distribution.png', dpi=150)
plt.show()
```



. Model — ConvNeXt-Tiny (SiLU | StochDepth | LayerNorm)

In [8]:
```
def replace_activation(model, src_cls, dst_fn):
    for name, module in model.named_children():
        if isinstance(module, src_cls):
            setattr(model, name, dst_fn())
        else:
            replace_activation(module, src_cls, dst_fn)

model = models.convnext_tiny(weights=models.ConvNeXt_Tiny_Weights.IMAGENE
in_features = model.classifier[2].in_features
model.classifier[2] = nn.Linear(in_features, NUM_CLASSES)

# Replace GELU → SiLU
replace_activation(model, nn.GELU, nn.SiLU)
print('Activation: GELU → SiLU')

model = model.to(DEVICE)
total_p     = sum(p.numel() for p in model.parameters())
trainable_p = sum(p.numel() for p in model.parameters() if p.requires_gra
print(f'Experiment : {EXP_NAME}')
print(f'Total params     : {total_p:,}')
print(f'Trainable params: {trainable_p:,}')
```

```
Downloading: "https://download.pytorch.org/models/convnext_tiny-983f1562.pth"
root/.cache/torch/hub/checkpoints/convnext_tiny-983f1562.pth
100%|██████████| 109M/109M [00:00<00:00, 189MB/s]
Activation: GELU → SiLU
Experiment : ConvNeXt-Tiny | SiLU | StochDepth | LayerNorm
Total params     : 27,825,511
Trainable params: 27,825,511
```

. Training Setup

In [9]:
```
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=LR_INIT, weight_decay=WEIG
```

```
def get_scheduler(optimizer, num_epochs, lr_init, lr_final):
    warmup = optim.lr_scheduler.LinearLR(optimizer, start_factor=0.01, en
    cosine = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_ep
    return optim.lr_scheduler.SequentialLR(optimizer, schedulers=[warmup,

scheduler = get_scheduler(optimizer, NUM_EPOCHS, LR_INIT, LR_FINAL)
```

. Training Loop

In [10]:

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total   += labels.size(0)
    return running_loss / total, correct / total

def evaluate(model, loader, criterion):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total   += labels.size(0)
    return running_loss / total, correct / total

history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc':
best_val_acc = 0.0

for epoch in range(NUM_EPOCHS):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterio
    val_loss,   val_acc   = evaluate(model, val_loader, criterion)
    scheduler.step()
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), SAVE_PATH)
    print(f'Epoch [{epoch+1:02d}/{NUM_EPOCHS}] '
          f'Train Loss: {train_loss:.4f} Acc: {train_acc:.4f} | '
          f'Val Loss: {val_loss:.4f} Acc: {val_acc:.4f}'
          + (' <- best' if val_acc == best_val_acc else ''))
```

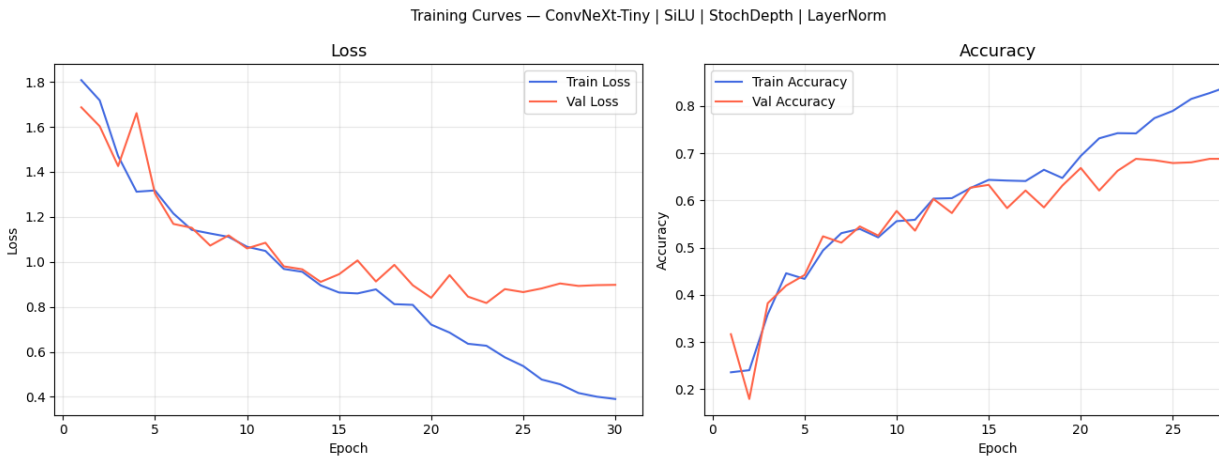
```
print(f'\nBest Val Accuracy: {best_val_acc:.4f}')
print(f'Saved: {SAVE_PATH}')
```

```
Epoch [01/30] Train Loss: 1.8076 Acc: 0.2357 | Val Loss: 1.6868 Acc: 0.3164 <
Epoch [02/30] Train Loss: 1.7177 Acc: 0.2402 | Val Loss: 1.6026 Acc: 0.1791
Epoch [03/30] Train Loss: 1.4704 Acc: 0.3585 | Val Loss: 1.4256 Acc: 0.3821 <
Epoch [04/30] Train Loss: 1.3117 Acc: 0.4455 | Val Loss: 1.6610 Acc: 0.4194 <
Epoch [05/30] Train Loss: 1.3173 Acc: 0.4336 | Val Loss: 1.3038 Acc: 0.4418 <
Epoch [06/30] Train Loss: 1.2149 Acc: 0.4938 | Val Loss: 1.1686 Acc: 0.5239 <
Epoch [07/30] Train Loss: 1.1421 Acc: 0.5306 | Val Loss: 1.1513 Acc: 0.5104
Epoch [08/30] Train Loss: 1.1262 Acc: 0.5395 | Val Loss: 1.0723 Acc: 0.5448 <
Epoch [09/30] Train Loss: 1.1110 Acc: 0.5216 | Val Loss: 1.1176 Acc: 0.5254
Epoch [10/30] Train Loss: 1.0670 Acc: 0.5559 | Val Loss: 1.0598 Acc: 0.5776 <
Epoch [11/30] Train Loss: 1.0485 Acc: 0.5589 | Val Loss: 1.0848 Acc: 0.5358
Epoch [12/30] Train Loss: 0.9684 Acc: 0.6037 | Val Loss: 0.9799 Acc: 0.6030 <
Epoch [13/30] Train Loss: 0.9558 Acc: 0.6047 | Val Loss: 0.9669 Acc: 0.5731
Epoch [14/30] Train Loss: 0.8955 Acc: 0.6261 | Val Loss: 0.9109 Acc: 0.6269 <
Epoch [15/30] Train Loss: 0.8636 Acc: 0.6435 | Val Loss: 0.9451 Acc: 0.6328 <
Epoch [16/30] Train Loss: 0.8597 Acc: 0.6420 | Val Loss: 1.0063 Acc: 0.5836
Epoch [17/30] Train Loss: 0.8777 Acc: 0.6410 | Val Loss: 0.9129 Acc: 0.6209
Epoch [18/30] Train Loss: 0.8119 Acc: 0.6648 | Val Loss: 0.9866 Acc: 0.5851
Epoch [19/30] Train Loss: 0.8089 Acc: 0.6474 | Val Loss: 0.8960 Acc: 0.6313
Epoch [20/30] Train Loss: 0.7211 Acc: 0.6942 | Val Loss: 0.8402 Acc: 0.6687 <
Epoch [21/30] Train Loss: 0.6857 Acc: 0.7315 | Val Loss: 0.9410 Acc: 0.6209
Epoch [22/30] Train Loss: 0.6359 Acc: 0.7424 | Val Loss: 0.8455 Acc: 0.6627
Epoch [23/30] Train Loss: 0.6272 Acc: 0.7419 | Val Loss: 0.8171 Acc: 0.6881 <
Epoch [24/30] Train Loss: 0.5759 Acc: 0.7742 | Val Loss: 0.8789 Acc: 0.6851
Epoch [25/30] Train Loss: 0.5368 Acc: 0.7897 | Val Loss: 0.8656 Acc: 0.6791
Epoch [26/30] Train Loss: 0.4771 Acc: 0.8150 | Val Loss: 0.8818 Acc: 0.6806
Epoch [27/30] Train Loss: 0.4561 Acc: 0.8274 | Val Loss: 0.9040 Acc: 0.6881 <
Epoch [28/30] Train Loss: 0.4172 Acc: 0.8414 | Val Loss: 0.8930 Acc: 0.6881 <
Epoch [29/30] Train Loss: 0.4007 Acc: 0.8458 | Val Loss: 0.8966 Acc: 0.6881 <
Epoch [30/30] Train Loss: 0.3905 Acc: 0.8548 | Val Loss: 0.8978 Acc: 0.6940 <

Best Val Accuracy: 0.6940
Saved: /content/drive/MyDrive/rice-convnext/05_SiLU_StochDepth_LayerNorm/
convnext_tiny_05_silu_stochdepth_layernorm.pt
```

. Training Curves

In [11]:

```
epochs_range = range(1, NUM_EPOCHS + 1)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(epochs_range, history['train_loss'], label='Train Loss', col
axes[0].plot(epochs_range, history['val_loss'],   label='Val Loss',   col
axes[0].set_title('Loss', fontsize=13)
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].legend(); axes[0].grid(alpha=0.3)
axes[1].plot(epochs_range, history['train_acc'], label='Train Accuracy',
axes[1].plot(epochs_range, history['val_acc'],   label='Val Accuracy',
axes[1].set_title('Accuracy', fontsize=13)
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy')
axes[1].legend(); axes[1].grid(alpha=0.3)
plt.suptitle(f'Training Curves — {EXP_NAME}', fontsize=11)
plt.tight_layout()
plt.savefig('training_curves.png', dpi=150)
plt.show()
```



. Evaluation on Test Set

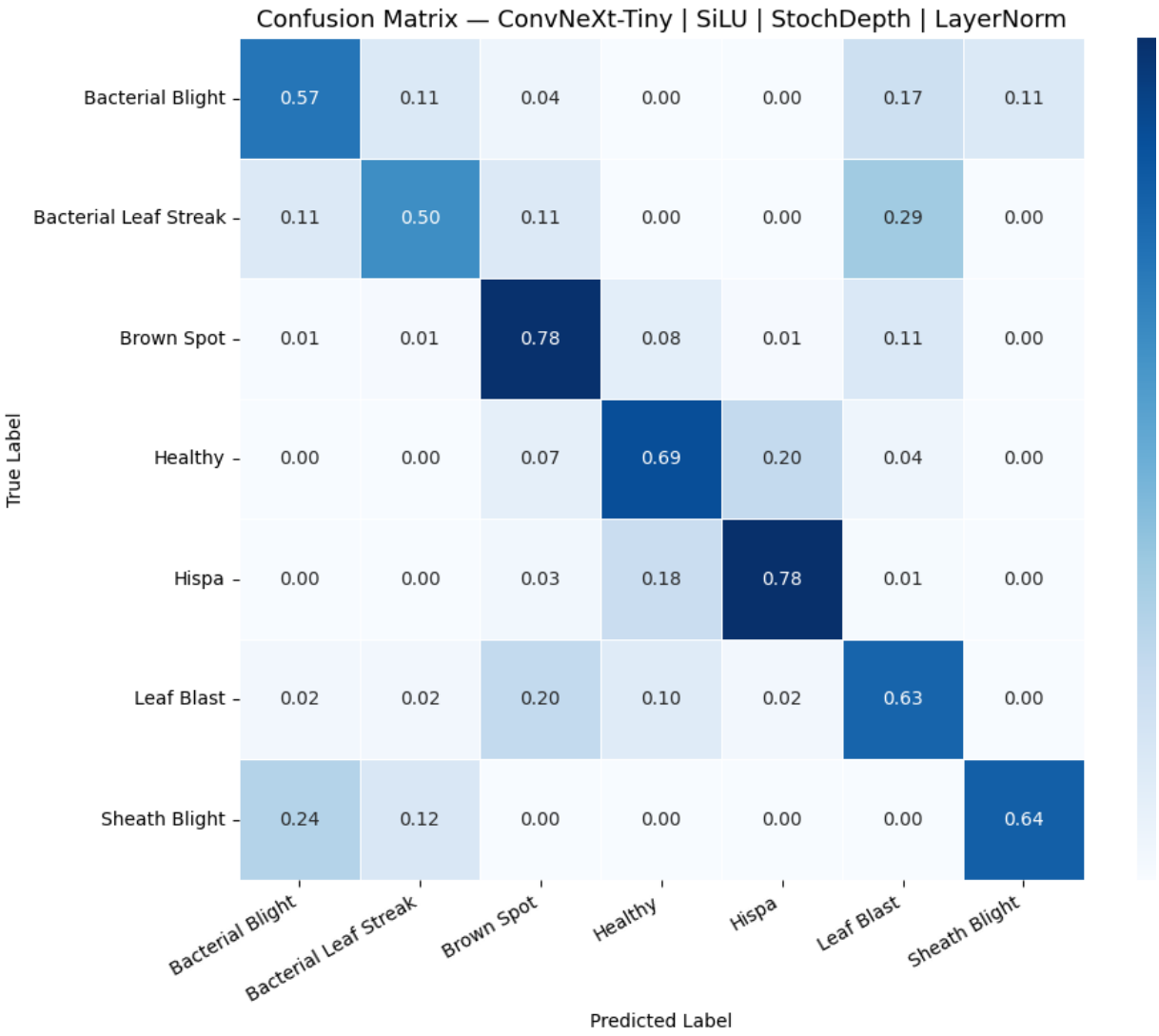
In [12]:
```
model.load_state_dict(torch.load(SAVE_PATH, map_location=DEVICE))
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(DEVICE)
        outputs = model(images)
        _, predicted = outputs.max(1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.numpy())

report = classification_report(all_labels, all_preds, target_names=CLASS_
report_df = pd.DataFrame(report).transpose()
print('=== Classification Report ===')
print(report_df.round(4).to_string())
report_df.to_csv('classification_report.csv')

summary = {
    'Model'     : ['ConvNeXt-Tiny'],
    'Activation': ['SiLU'],
    'Dropout'   : ['StochDepth'],
    'Norm'      : ['LayerNorm'],
    'Accuracy'  : [report['accuracy']],
    'Precision' : [report['weighted avg']['precision']],
    'Recall'    : [report['weighted avg']['recall']],
    'F1-Score'  : [report['weighted avg']['f1-score']],
}
summary_df = pd.DataFrame(summary)
print('\n=== Summary ===')
print(summary_df.round(4).to_string(index=False))
summary_df.to_csv('summary.csv', index=False)
```

```
=== Classification Report ===
                      precision   recall  f1-score   support
Bacterial Blight         0.7045  0.5741    0.6327   54.0000
Bacterial Leaf Streak    0.5000  0.5000    0.5000   28.0000
Brown Spot               0.7414  0.7771    0.7588  166.0000
Healthy                  0.7188  0.6928    0.7055  166.0000
Hispa                    0.6935  0.7818    0.7350  110.0000
Leaf Blast               0.6417  0.6260    0.6337  123.0000
Sheath Blight            0.7273  0.6400    0.6809   25.0000
accuracy                 0.6964  0.6964    0.6964    0.6964
macro avg                0.6753  0.6560    0.6638  672.0000
weighted avg             0.6962  0.6964    0.6950  672.0000

=== Summary ===
        Model Activation    Dropout      Norm  Accuracy  Precision  Recall  F
Score
ConvNeXt-Tiny       SiLU StochDepth LayerNorm    0.6964     0.6962  0.6964
0.695
```

. Confusion Matrix

In [13]:

```
cm = confusion_matrix(all_labels, all_preds)
cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
plt.figure(figsize=(10, 8))
sns.heatmap(cm_norm, annot=True, fmt='.2f',
            xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES,
            cmap='Blues', linewidths=0.5)
plt.title(f'Confusion Matrix — ConvNeXt-Tiny | SiLU | StochDepth | LayerN
plt.ylabel('True Label'); plt.xlabel('Predicted Label')
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()
```



. Save All to Drive

In [14]:
```
local_files = ['training_curves.png', 'confusion_matrix.png',
               'classification_report.csv', 'summary.csv', 'class_distrib
for fname in local_files:
    if os.path.exists(fname):
        shutil.copy(fname, f'{DRIVE_DIR}/{fname}')

all_outputs = [
    (f'{DRIVE_DIR}/convnext_tiny_05_silu_stochdepth_layernorm.pt',
    (f'{DRIVE_DIR}/training_curves.png',              'Training curves'),
    (f'{DRIVE_DIR}/confusion_matrix.png',             'Confusion matrix')
    (f'{DRIVE_DIR}/classification_report.csv',        'Per-class metrics'
    (f'{DRIVE_DIR}/summary.csv',                      'Summary table'),
]
print(f'=== Output Files: {DRIVE_DIR} ===')
for fpath, desc in all_outputs:
    exists = '✅' if os.path.exists(fpath) else '❌'
    print(f'{exists}  {os.path.basename(fpath):<45} {desc}')
```

```
=== Output Files: /content/drive/MyDrive/rice-convnext/05_SiLU_StochDepth_Lay
===
✅  convnext_tiny_05_silu_stochdepth_layernorm.pt Model weights
✅  training_curves.png                           Training curves
✅  confusion_matrix.png                          Confusion matrix
✅  classification_report.csv                     Per-class metrics
✅  summary.csv                                   Summary table
```